

raft-kv: The Deep Dive

A Distributed Key-Value Store with Raft Consensus,
Deterministic Simulation, and a Linearizability Checker

Explainer document – generated for Salman Adnan’s portfolio

This document is a working explainer, not published website content. Every fact it states comes from the project’s source code, tests, and README. Nothing here is invented; where something is inferred rather than explicitly stated in the repository, it is labeled “inferred.”

Contents

1	What This Project Is	3
2	Project Layout	4
3	Foundational Concepts	4
4	The Raft Consensus Core (raft/)	5
4.1	Roles and the state machine	5
4.2	Leader election	6
4.3	Becoming leader, and the no-op entry	7
4.4	Log replication	7
4.5	Commit and apply	8
4.6	Persistence discipline	8
4.7	Snapshots and <code>InstallSnapshot</code>	9
4.8	Membership changes: single-server (not joint consensus)	10
4.9	Linearizable reads without the log: <code>ReadIndex</code>	10
5	The Replicated State Machine (kv/)	11
5.1	Client sessions and exactly-once semantics	11
6	Two Transports Behind One Env (transport/, sim/)	12
6.1	Production: <code>TcpTransport</code> and <code>AsyncioEnv</code>	12
6.2	Simulation: <code>SimClock</code> , <code>SimNetwork</code> , <code>SimEnv</code>	12
6.3	Determinism as plumbing	13
7	The Chaos Harness (sim/harness.py)	13
7.1	The nemesis	13
7.2	Continuous safety invariants	14
7.3	Simulated clients	15
8	The Linearizability Checker (checker/)	15

8.1	The problem	15
8.2	Wing & Gong search with Lowe's memoization	16
8.3	Guarding against a checker that always says yes	17
9	Wiring It All Together: server.py, client.py, cli/	17
9.1	RaftServer: one real cluster node	17
9.2	RaftClient: leader-hint following and retries	17
9.3	The CLI	18
10	Testing and Verification	18
10.1	Test suites	18
10.2	A noteworthy testing challenge: Figure 7 without real elections	19
10.3	Verification results actually observed	19
11	Honest Limits, Challenges, and Lessons Learned	19
11.1	Stated honest limits	19
11.2	Real challenges hit during development	20
11.3	What the author says they learned	20
11.4	What the author says they would change	21
12	Glossary	21

1 What This Project Is

raft-kv is a distributed key-value store written from scratch in Python 3.12, with zero runtime dependencies. “Distributed” means the data lives on several independent machines (or processes) called **nodes**, rather than one, so the system keeps working if some nodes crash. To make several independent nodes agree on one sequence of operations despite crashes, message loss, and network partitions, the project implements the **Raft consensus algorithm** from scratch, following:

- Diego Ongaro and John Ousterhout, *In Search of an Understandable Consensus Algorithm (Extended Version)*, USENIX ATC 2014 – the core election, replication, and safety rules;
- Diego Ongaro’s PhD dissertation, *Consensus: Bridging Theory and Practice* (Stanford, 2014) – log compaction (snapshots) and single-server membership changes.

No existing Raft or consensus library is used anywhere in the code, and the linearizability checker described later is an independent implementation, not a port of an existing tool (the README explicitly notes it is *not* a port of “porcupine,” a well-known Go linearizability checker).

Why consensus is hard to trust from ordinary tests

A key claim the author makes, and the whole second half of this document backs up: consensus bugs live in *message interleavings* – the precise, rare orderings in which messages arrive, get delayed, get lost, or race against timeouts and crashes. Running a cluster on a laptop and clicking around by hand will essentially never reproduce the interleaving that breaks the algorithm. The project’s answer, discussed in Section 6, is to make an entire cluster’s behavior a deterministic function of a single integer (a **seed**), so that thousands of adversarial schedules can be generated and replayed exactly.

The central design idea: sans-IO

Every piece of this project follows from one decision, described in the project’s own architecture notes as “sans-IO” (without I/O). The class that implements Raft, **RaftNode**, never calls **asyncio**, never reads the wall clock, never opens a socket, and never calls Python’s global **random** module. It only reacts to three kinds of stimuli – an incoming message, a timer it previously scheduled, or a local method call like “propose this write” – and it only produces effects through two small interfaces it is handed at construction time: an **Env** (environment: current time, schedule a timer, send a message, a random number generator) and a **Storage** (persist term/vote/log/snapshot).

Because **RaftNode** is blind to *how* time passes and messages travel, the exact same class can run:

- over a real TCP network, driven by **AsyncioEnv** (real asyncio clock and sockets), or
- inside a deterministic simulator, driven by **SimEnv** (a virtual clock and an in-process fake network with seeded delay, loss, duplication, and partitions).

Every bug the simulator finds is therefore a bug in the actual production code path, not a bug in a separate test double that merely resembles it. This idea recurs throughout the document and is the single most important architectural fact about the project.

2 Project Layout

The source lives under `src/raftkv/`, organized by responsibility. Figure 1 shows the directory tree; the sections that follow walk each package in the order a reader would need to understand them.

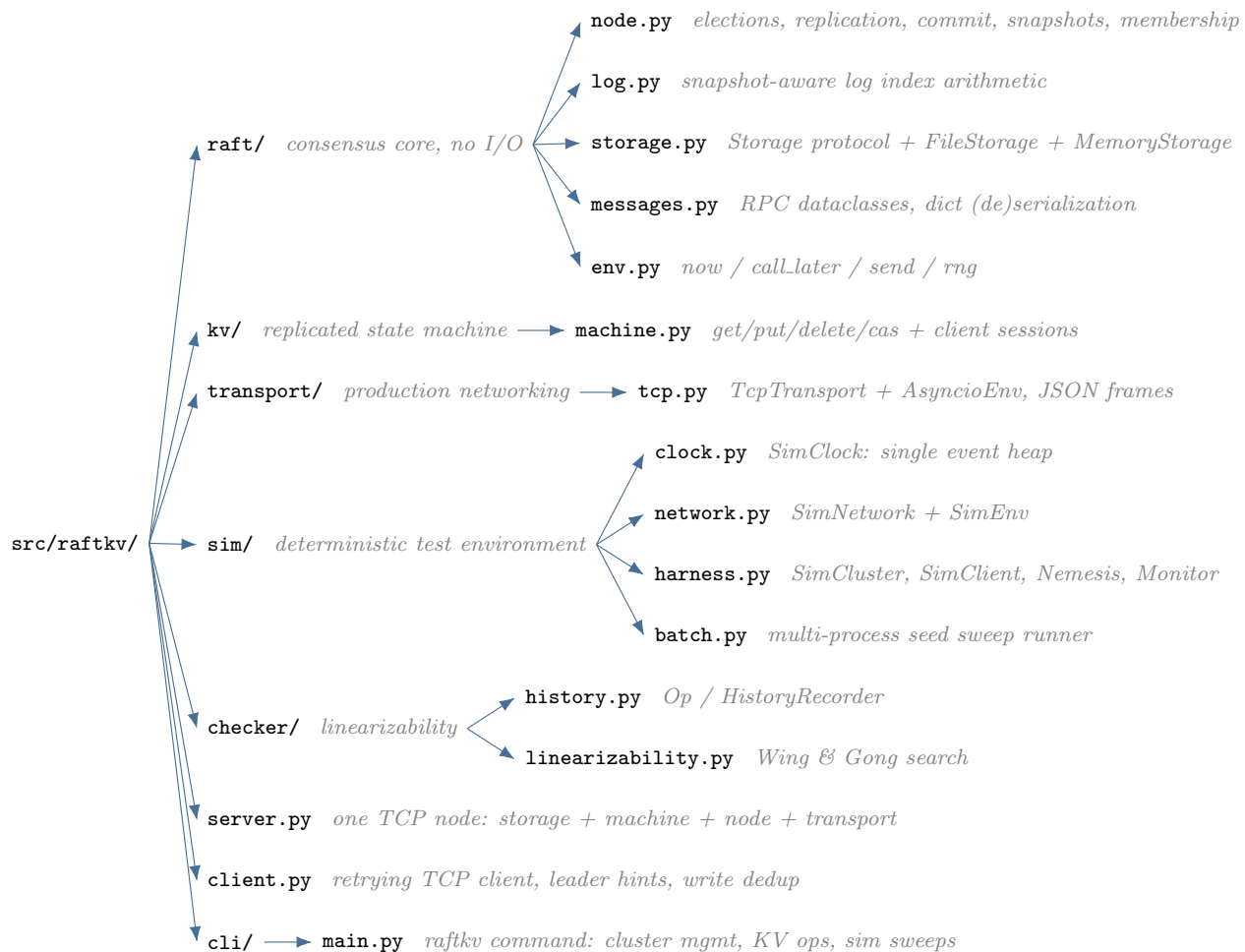


Figure 1: Package layout of `src/raftkv/`. Roughly 3,700 lines of Python across source and tests.

Two more directories matter: `tests/` (ten test modules, discussed in Section 10) and `docs/` (`ARCHITECTURE.md` and `TESTING.md`, the author’s own design notes, which this document draws on and cross-checks against the code).

3 Foundational Concepts

Before the Raft-specific mechanics, four terms recur throughout this document and are defined once here.

Consensus

Getting a group of independent nodes to agree on a single, ordered sequence of values (here: KV operations) even when some nodes crash, restart, or cannot currently talk to each other, and even when messages are delayed, lost, or duplicated.

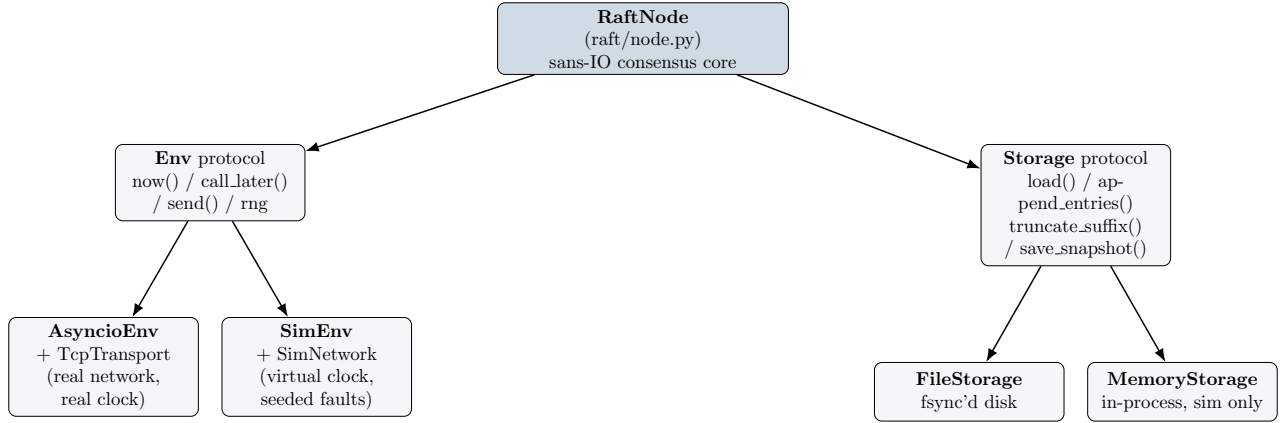


Figure 2: The sans-IO seam. **RaftNode** is identical in both deployments; only the **Env** and **Storage** implementations swapped underneath it change what “the network” and “the disk” mean.

Replicated log

An append-only, ordered list of **entries** that every node keeps a copy of. Raft’s job is to keep every node’s log identical up to the point that has been **committed** (safely agreed upon by a majority). The key-value data itself is derived by **applying** committed log entries in order to a state machine.

Quorum / majority

For N nodes, a majority is $\lfloor N/2 \rfloor + 1$. Any two majorities out of the same N nodes must share at least one node. Raft’s safety proofs lean on this overlap: whatever one majority agreed to, any later majority is guaranteed to have at least one member who already knows about it.

Linearizability

A correctness property for concurrent operations on shared data: the system must behave as if every operation took effect atomically at some single instant between when the client sent it and when the client received its response, and those instants form a sequence consistent with the actual results observed. Section 8 covers how this is checked mechanically.

4 The Raft Consensus Core (raft/)

4.1 Roles and the state machine

Every node is in exactly one of three roles at a time, defined in `raft/node.py` as the `Role` enum: **Follower**, **Candidate**, and **Leader**. Figure 3 shows the transitions, all implemented in **RaftNode**.

Timers

Every node runs an **election timer**: a randomized delay, uniformly chosen between `election_timeout` and $2 * \text{election_timeout}$ (`_reset_election_timer`), that fires `_on_election_timeout` if not cancelled first. Randomizing the timeout (rather than using a fixed value) is what keeps two nodes from perpetually calling elections at the same instant and splitting the vote forever – whichever node’s random delay happens to be shortest usually gets to campaign, and collect votes, before anyone else’s timer fires.

A **Leader** additionally runs a fixed-period **heartbeat timer** (`heartbeat_interval`) that calls

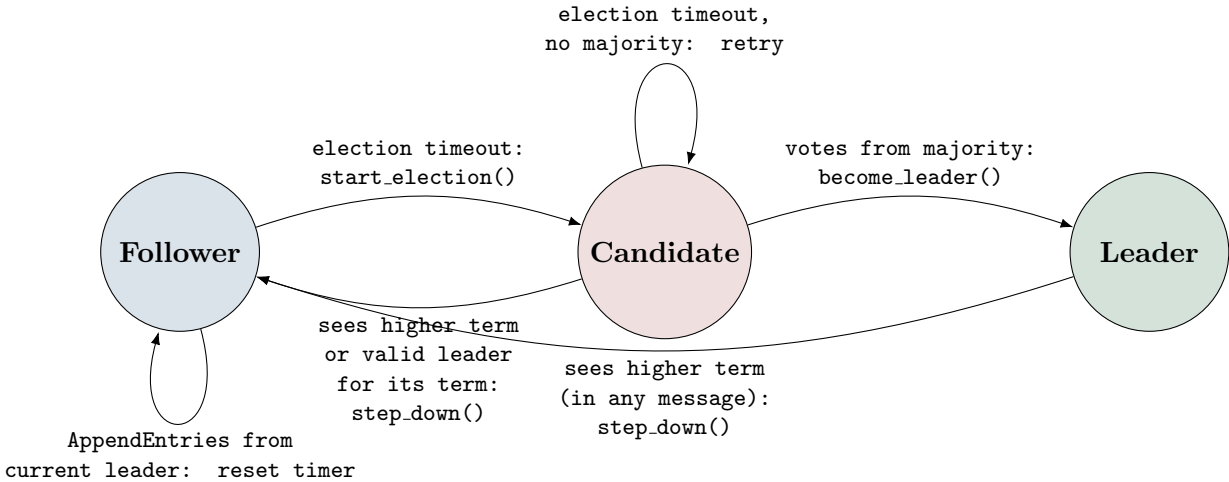


Figure 3: Raft’s role state machine as implemented in `RaftNode`. A node also self-transitions `Candidate`→`Leader` instantly if it already holds a majority (e.g. a single-node cluster) inside `_start_election`.

`_broadcast_append` – sending (possibly empty) `AppendEntries` to every peer, both to replicate new entries and to prove to followers that the leader is still alive (resetting their election timers).

4.2 Leader election

`_start_election` implements the candidate’s side:

1. Increment `current_term` and vote for self (`voted_for = self.id`). This is persisted to disk *before* anything else happens (`_persist_term_vote`) – if the node crashes right here and restarts, it must not accidentally vote twice in the same term.
2. If a majority is already held (only possible in a single-node cluster, since a node always votes for itself), skip straight to `_become_leader`.
3. Otherwise send `RequestVote` to every peer, carrying the candidate’s last log index and term.

A receiving node (`_on_request_vote`) grants its vote only if *all* of these hold:

- the request’s term equals the receiver’s current term (a lower term is rejected outright; a higher term causes the receiver to step down to follower *first*, in `handle_message`, before this check runs, so the comparison is always against the now-updated term),
- the receiver has not already voted for someone else this term (`voted_for in (None, candidate_id)`), and
- the candidate’s log is at least as up to date as the receiver’s, compared as the pair (`last_log_term`, `last_log_index`) – this is the paper’s Section 5.4.1 rule, and it is what prevents a node whose log is missing committed entries from ever becoming leader.

Granting a vote is itself persisted (`_persist_term_vote`) before the reply is sent, and it resets the receiver’s own election timer (a node that just voted for someone should give that candidate a chance to win before starting its own competing election).

`_on_request_vote_reply` tallies votes only while still a candidate in the same term the vote was

requested for (stale replies from an old term, or replies after the node already moved on, are ignored); once a majority of votes (restricted to nodes in the *current* config, guarding against a stale vote from a since-removed node) is reached, `_become_leader` runs.

4.3 Becoming leader, and the no-op entry

`_become_leader` resets the leader’s per-follower bookkeeping (`next_index`, `match_index`: explained in Section 4.4) and then does something subtle but essential: it appends a **no-op entry** to its own log in its own new term, *before* sending anything to followers.

Why the no-op matters (Section 5.4.2 of the paper). Raft never lets a leader advance the commit index based purely on counting replicas of an entry from an *older* term, even once a majority holds it – doing so can be unsafe (the paper shows a scenario where such an entry can later be overwritten by a future leader despite looking “committed”). A leader may only conclude an entry is committed by counting replicas of an entry from its *own current term*. But old, truly-committed entries still need to become visible eventually. The fix: every new leader immediately appends a no-op entry in its own term; once *that* entry commits (which pulls every older entry along with it, because commit index only moves forward as one number), all the older entries commit too. `_maybe_advance_commit` enforces the own-term restriction directly:

```
candidate = matches[len(matches) - self._majority()]
if candidate > self.commit_index and self.log.term_at(candidate) == self.current_term:
    self._set_commit_index(candidate)
```

4.4 Log replication

`next_index[peer]` is the leader’s guess at the next log index that peer needs; `match_index[peer]` is the highest index the leader *knows* is replicated there. `_send_append` sends everything in the log from `next_index[peer]` onward, tagged with `prev_log_index/prev_log_term` (the entry immediately before that range) so the follower can verify continuity.

Figure 4 traces one successful replication round end to end.

Conflict detection in detail. `_on_append_entries` first checks whether the follower even has an entry at `prev_log_index`; if not, it asks for everything (`conflict = last_index + 1`). If it has an entry there but the term differs, it walks backward while the term stays the same, so the conflict index it reports is the *first* index of that conflicting term – letting the leader skip the entire conflicting term in one round trip rather than decrementing one index per round trip (an $O(n)$ -round approach the original paper allows but discourages).

Truncation on conflict. When the leader’s entries do arrive and one of them has a different term than what the follower already has at that index, the follower truncates its log from that point (`log.truncate_suffix`, mirrored to disk with `storage.truncate_suffix`) before appending the leader’s version. This is always safe because whatever was truncated was, by definition, never committed (an entry only commits once a leader confirms a majority holds the identical entry at that index in its own term).

Snapshot-aware replication. If `prev_index` falls at or below the follower’s snapshot point, the leader’s message is truncated to only the entries the follower doesn’t already have via its snapshot,

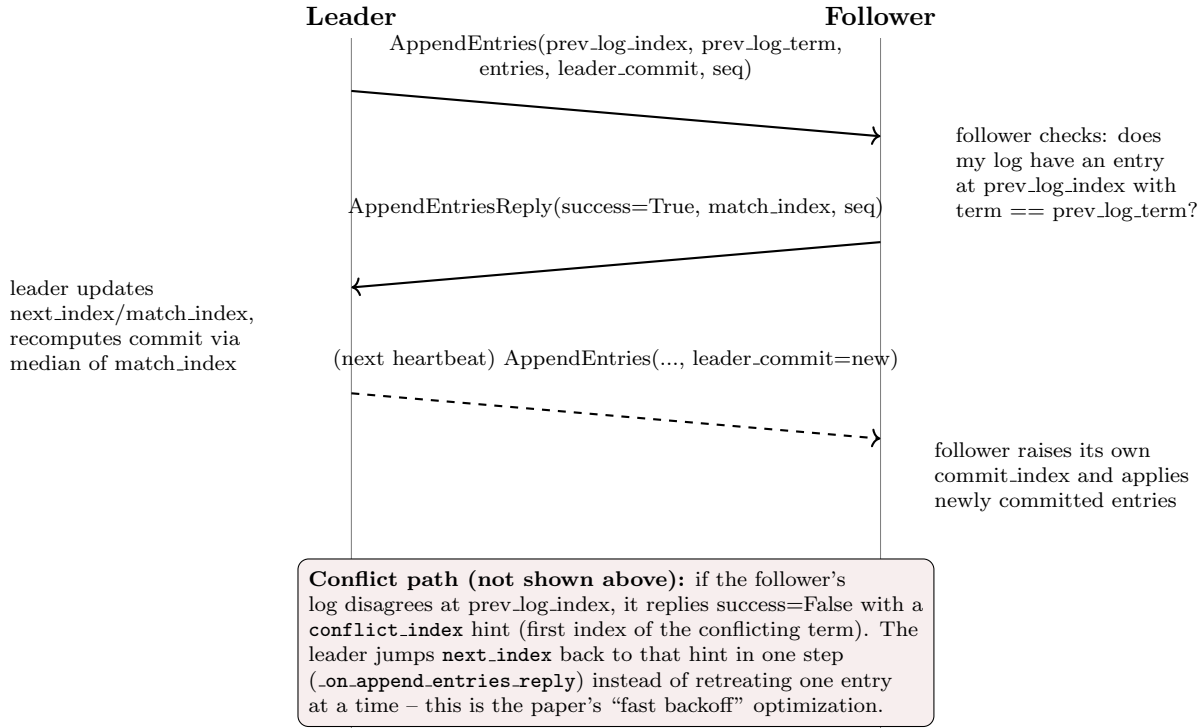


Figure 4: One AppendEntries round trip, and the fast-backoff conflict path.

and `prev_index/prev_term` are substituted with the snapshot's own index/term – anything at or below a snapshot is by definition committed and therefore guaranteed to match.

4.5 Commit and apply

`_maybe_advance_commit` (leader only) computes, across the *current cluster config* (not a fixed set – membership can change, Section 4.8), the median of `match_index` values (treating the leader's own log as always fully matched). That median is exactly the highest index held by a majority. If that index is higher than the current commit index *and* belongs to the leader's own current term, commit index advances to it.

`_set_commit_index` then calls `_apply_committed`, which walks `last_applied` forward to the new `commit_index`, applying each entry's command to the state machine (for "command" entries only; "noop" and "config" entries are not handed to the KV state machine) and notifying `apply_listeners`. This is also where `_maybe_snapshot` is triggered once enough entries have accumulated past the last snapshot point.

4.6 Persistence discipline

Two `Storage` implementations satisfy one `Storage` protocol (`raft/storage.py`):

MemoryStorage

An in-process Python object holding the same fields a real disk would. Used only by the simulator. It deliberately *survives* a simulated node crash (the harness keeps the `MemoryStorage` object alive while discarding everything else about the node), modeling “the disk kept its data, the process restarted.”

FileStorage

Real on-disk persistence, used by every real TCP node, made of three files per node:

- `hardstate.json` – `current_term` and `voted_for`. Written via **atomic write**: data goes to a temp file, is `fsync`'d (flushed all the way to the physical disk, not just the OS page cache), then `rename`'d over the real path – a rename is atomic on POSIX filesystems, so a crash mid-write leaves either the old file or the new one, never a half-written one.
- `log.bin` – an append-only file of log entries. Each record is [4-byte big-endian length][4-byte CRC32 checksum][JSON payload]. On load, the file is scanned sequentially; the moment a record's declared length runs past the end of the file, or its checksum doesn't match, that is a **torn write** (a crash that happened mid-`write()`), and everything from that point on is discarded and the file is truncated back to the last good record. This is safe precisely because Raft's rule is never to acknowledge a message (a vote, an `AppendEntries` success) whose effects (the vote or the entries) have not yet been `fsync`'d – so a torn, discarded record was never something any other node was told about.
- `snapshot.json` – the latest snapshot's metadata and the state machine's serialized data, also atomically written. After a snapshot is taken, `log.bin` is entirely rewritten to contain only the entries after the snapshot point.

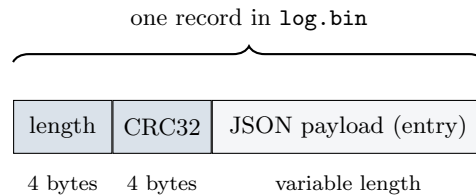


Figure 5: Every appended log entry is framed with a length and a checksum so a torn tail from a crash is detected on the next load and discarded.

4.7 Snapshots and InstallSnapshot

As the log grows, replaying it from the start on every restart, and keeping every historical entry on every node forever, becomes wasteful. **Log compaction** solves this: once `last_applied` – `snapshot_index` passes `snapshot_threshold`, `maybe_snapshot` asks the state machine for a serialized snapshot of its entire current data (KV data, sessions, and current membership config – see `KVMachine.snapshot`), records the entry index and term the snapshot covers, and asks `log.compact_to` to discard the log entries the snapshot now makes redundant.

If a follower is so far behind that the leader no longer has the entries it needs (`next_index[follower] <= log.snapshot_index`), the leader instead sends the whole snapshot as one `InstallSnapshot` message. The follower installs it (`on_install_snapshot`), handling two cases: if its own log already independently extends past the snapshot point with a matching term, it keeps that suffix (`log.compact_to`); otherwise its log is entirely too far diverged and is replaced outright with a fresh, empty log starting at the snapshot point.

The README is explicit about an honest limitation here: the snapshot travels as a single message, not chunked, which is fine at the scale this project targets (a handful of nodes, localhost) but would not scale to snapshots of realistic production size.

4.8 Membership changes: single-server (not joint consensus)

Raft clusters sometimes need to change which nodes are members (adding or removing a server). The naive approach – just tell everyone the new list – is unsafe: during the transition, some nodes might compute majorities against the old member list and others against the new one, and those two majorities might not overlap, letting two different leaders win simultaneously (a **split brain**).

This project implements the dissertation’s **single-server** approach: membership changes one server at a time, expressed as an ordinary log entry of kind **"config"** carrying the full new node list. Because any two configurations that differ by exactly one member are guaranteed to have overlapping majorities (a simple counting argument: if $\lceil (n+1)/2 \rceil$ out of $n+1$ nodes and $\lceil n/2 \rceil$ out of n nodes are each a majority, they cannot be disjoint), there is never a moment where two disjoint majorities could both elect a leader. Config changes take effect immediately on being *appended* to the log (not only once committed) – `_refresh_config` runs right after any append or truncation that touches the log – and a second config change is rejected while one is still uncommitted (`_config_change_pending`), preventing overlapping changes.

A leader that removes itself keeps leading (so the removal can actually commit) until that config entry is applied, at which point `_apply_committed` notices its own id is no longer in the new config and calls `_step_down`. A node just added starts with an empty initial config and never campaigns for votes on its own (checked in `_on_election_timeout`: `self.id not in self.config`) – it purely waits to be told the cluster’s makeup by the leader’s log or a snapshot.

Not implemented (stated candidly in the README and ARCHITECTURE.md): **joint consensus** (the paper’s more general, two-phase membership mechanism that permits arbitrary jumps between configurations, at the cost of a more complex “both quorums must agree” transitional state), pre-vote, leader stickiness, and automatic catch-up rounds before a newly added, far-behind server counts toward the commit quorum.

4.9 Linearizable reads without the log: ReadIndex

A naive way to serve a `get` would be to run it through the log like a write, but that wastes a disk fsync and a round trip on an operation that doesn’t change anything. The alternative implemented here is the **ReadIndex** protocol (`RaftNode.read_index`), which serves reads locally but only after proving current leadership:

1. The leader may not answer a read at all until an entry from its *own* current term has committed (the no-op from Section 4.4 again) – otherwise its own `commit_index` might not even reflect everything that is truly durable yet, from the Section 5.4.2 restriction. If that hasn’t happened, the read is registered with no read index yet and is completed automatically the moment `_set_commit_index` notices the first own-term commit (see `PendingRead.read_index`, set inside `_set_commit_index`).
2. Once available, the read captures `read_index = commit_index` at that instant.
3. The leader broadcasts a heartbeat round tagged with a sequence number and waits for acknowledgment from a majority of the *current* config for that exact round – proof that it is still recognized as leader by enough nodes right now, not just when it last checked.
4. Once acknowledged *and* `last_applied >= read_index` (the local state machine has actually caught up to that point), the read is served from local memory.

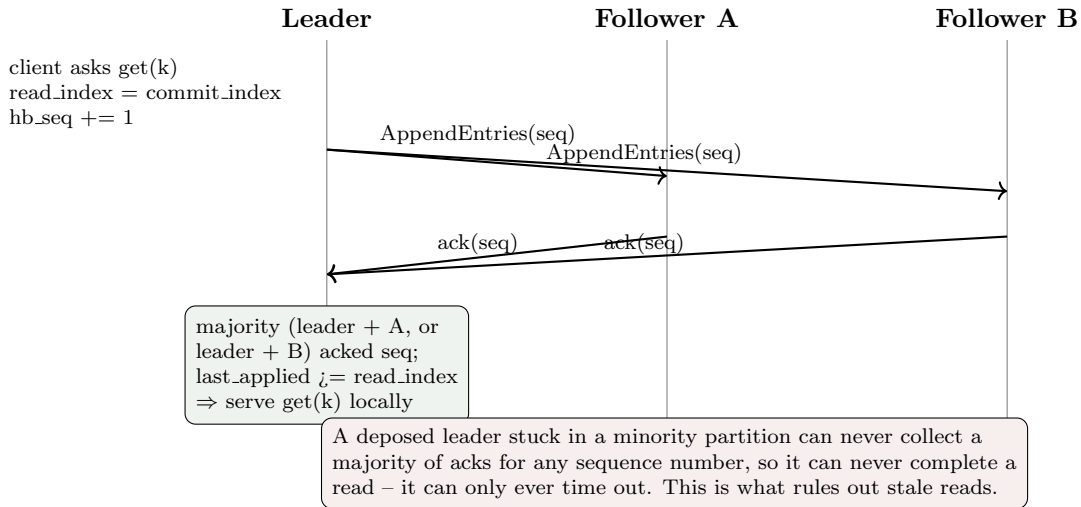


Figure 6: ReadIndex: a read is answered locally, but only after a heartbeat round proves the leader still commands a majority.

Why not leader leases instead? ARCHITECTURE.md explicitly discusses the alternative: a **leader lease** lets a leader serve reads locally for a bounded time after any successful heartbeat quorum, with no per-read round trip, as long as clocks across machines are assumed not to drift apart by more than some bound. Leases are faster, but their safety depends on that clock-drift assumption holding in practice. ReadIndex costs one heartbeat round per read (or one round shared across several concurrent reads, since the sequence number window is shared – see `_broadcast_append` calling `_check_pending_reads` for every batch) but needs no clock assumptions at all, which the author chose deliberately for a project whose whole point is testable correctness rather than raw speed.

5 The Replicated State Machine (kv/)

KVMachine (kv/machine.py) is what committed log entries are actually applied to. It supports four operations:

- `put(key, value)` – unconditional write.
- `delete(key)` – removes a key; result reports whether it existed.
- `cas(key, expected, value)` – **compare-and-swap**: only writes `value` if the key’s current value equals `expected` (with `None`/– meaning “key absent”); reports whether the swap happened and what the value was beforehand. This is the building block that lets a client do atomic read-modify-write without a distributed lock.
- `get(key)` – served entirely outside the log via ReadIndex (Section 4.9); `KVMachine.get` is a plain dictionary lookup once the caller has already established it is safe to read.

5.1 Client sessions and exactly-once semantics

Every write command optionally carries a `(client, seq)` pair. The state machine keeps, per `client` id, the highest `seq` it has applied and the result it produced (`self.sessions`). When `apply` sees a command whose `seq` is less than or equal to the cached one for that client, it returns the *cached* result without re-executing the command at all.

This matters because the network model here is only **at-least-once** at the transport layer: a client that doesn't get a response (timeout, lost reply, leader crash mid-request) cannot tell whether its write actually took effect, so the only safe thing to do is retry – but a naive retry of, say, a `cas` could apply it a second time and silently corrupt state (swap twice instead of once). Because the session table is itself part of the *replicated* state (it rides inside every snapshot, per `KVMachine.snapshot/restore`), a retried write that lands on a brand new leader after a failover still finds the original session entry and returns the original cached result – upgrading the transport's at-least-once delivery into application-level **exactly-once** apply semantics. The README documents this being specifically tested: a `cas` retried across a simulated leader crash returns the original “swapped” result instead of re-executing (and failing, since the precondition would no longer hold) a second time.

The honest cost, stated in both the README and ARCHITECTURE.md: sessions never expire, so `self.sessions` grows without bound as distinct client ids are seen – an accepted memory leak at this project's scale. A production system would agree on session expiration through the log itself (a TTL that all nodes apply identically), which this project does not implement.

6 Two Transports Behind One Env (transport/, sim/)

6.1 Production: TcpTransport and AsyncioEnv

`transport/tcp.py` implements a simple length-prefixed JSON framing: every frame on the wire is a 4-byte big-endian length followed by that many bytes of JSON (`read_frame/write_frame`). One listening TCP port per node carries both Raft peer-to-peer traffic and client request traffic; the first field of a decoded frame (`"kind": "hello", "raft",` or a client request) routes it.

Sends to peers are **fire-and-forget**: `TcpTransport.send` pushes a message onto a per-destination in-memory queue (`asyncio.Queue`, capacity 256) drained by a dedicated `_peer_writer` task that lazily opens (and reopens, on failure) a persistent connection to that peer. If the queue is full, the new message is silently dropped – this is deliberate, mirroring Raft's own tolerance for a lossy network: a dropped heartbeat or `AppendEntries` is simply resent on the next heartbeat tick. `AsyncioEnv` is the thin adapter that gives `RaftNode` its `now()/call_later()/send()` using the real `asyncio` event loop and this transport.

6.2 Simulation: SimClock, SimNetwork, SimEnv

The simulated stack (`sim/clock.py`, `sim/network.py`) exists to make the entire multi-node system a deterministic function of one integer seed, so that adversarial schedules can be searched cheaply and any failure can be replayed exactly.

SimClock

Holds one **binary heap** (a data structure that always gives you the smallest of the items you've inserted, in $O(\log n)$ time – exactly what you need to always process the next-soonest scheduled event) of `SimTimer` objects ordered by (`when`, `sequence-number`). *Every* source of activity in the simulator – election timers, heartbeats, message deliveries, nemesis actions, client retries – is one of these timers. `step()` pops and fires the single soonest one; `run_until(deadline)` drains the heap up to a virtual time bound. There is no real sleeping anywhere: 12 virtual seconds of simulated cluster activity execute in roughly 0.1 real seconds, per the README's measured throughput (5000 seeds in 84.7s, meaning each seed – 12 virtual seconds of 3-node cluster activity – runs in about 17ms of wall time).

SimNetwork

Implements the same `send/deliver` contract a real network would, but every delivery is scheduled as a `SimClock` event with a delay drawn uniformly from `[min_delay, max_delay]` using a seeded `random.Random`. Independently, each send has a `drop_prob` chance of being dropped entirely and a `dup_prob` chance of being delivered twice. **Partitions** (sets of nodes that currently cannot reach each other) are recorded as a set of node-pairs and checked *at delivery time, not at send time* – so a message that was “in flight” when a partition formed is dropped exactly as it would be if a real TCP connection died mid-transfer.

SimEnv

The per-node adapter satisfying the same `Env` protocol as `AsyncioEnv`, but backed by the shared `SimClock/SimNetwork` and a per-node `random.Random`. It additionally supports `timer_scale` (models clock skew: a node whose timers are scaled by, say, 1.8 fires its election timeout 80% later than nominal, simulating a slow local clock) and `kill()`, which invalidates every timer the node currently has outstanding by bumping an internal epoch counter that each scheduled callback checks before running – this is how a simulated crash instantly silences a node without having to track down and cancel every individual timer object.

6.3 Determinism as plumbing

The README calls out, as a specific lesson learned, that bit-for-bit determinism required treating randomness itself as an explicit, threaded-through resource rather than an ambient global. Concretely, `run_simulation` (in `sim/harness.py`) takes one `root random.Random(seed)` and fans it out:

```
root = random.Random(cfg.seed)
network = SimNetwork(clock, random.Random(root.getrandbits(64)), ...)
cluster_rng = random.Random(root.getrandbits(64))
nemesis = Nemesis(cluster, random.Random(root.getrandbits(64)), cfg, ...)
clients = [SimClient(..., random.Random(root.getrandbits(64)), ...) for i in range(cfg.
    n_clients)]
```

and each node, in turn, gets its own child `random.Random` inside `SimCluster.start_node`. Every event that ever happens goes through the single `(time, sequence)` ordered heap in `SimClock`, and partitions are resolved at delivery time as noted above. A dedicated test asserts that running the identical seed twice produces an identical, fingerprint-equal history; the README notes this test caught two real bugs during development where code accidentally relied on Python dictionary/set iteration order (“ambient ordering”) instead of the seeded RNG.

7 The Chaos Harness (`sim/harness.py`)

The harness entry point, `run_simulation`, taking one `SimConfig(seed=N)`, is the single entry point that runs one full, seeded chaos scenario. Figure 7 sketches the moving parts and Table 1 lists the default scenario shape.

7.1 The nemesis

`Nemesis._act` fires repeatedly (mean interval `partition_interval`, drawn from an exponential distribution – the standard way to model “random events at some average rate”) during the first 70% of the run (`chaos_deadline = duration * 0.7`), picking one action per firing:

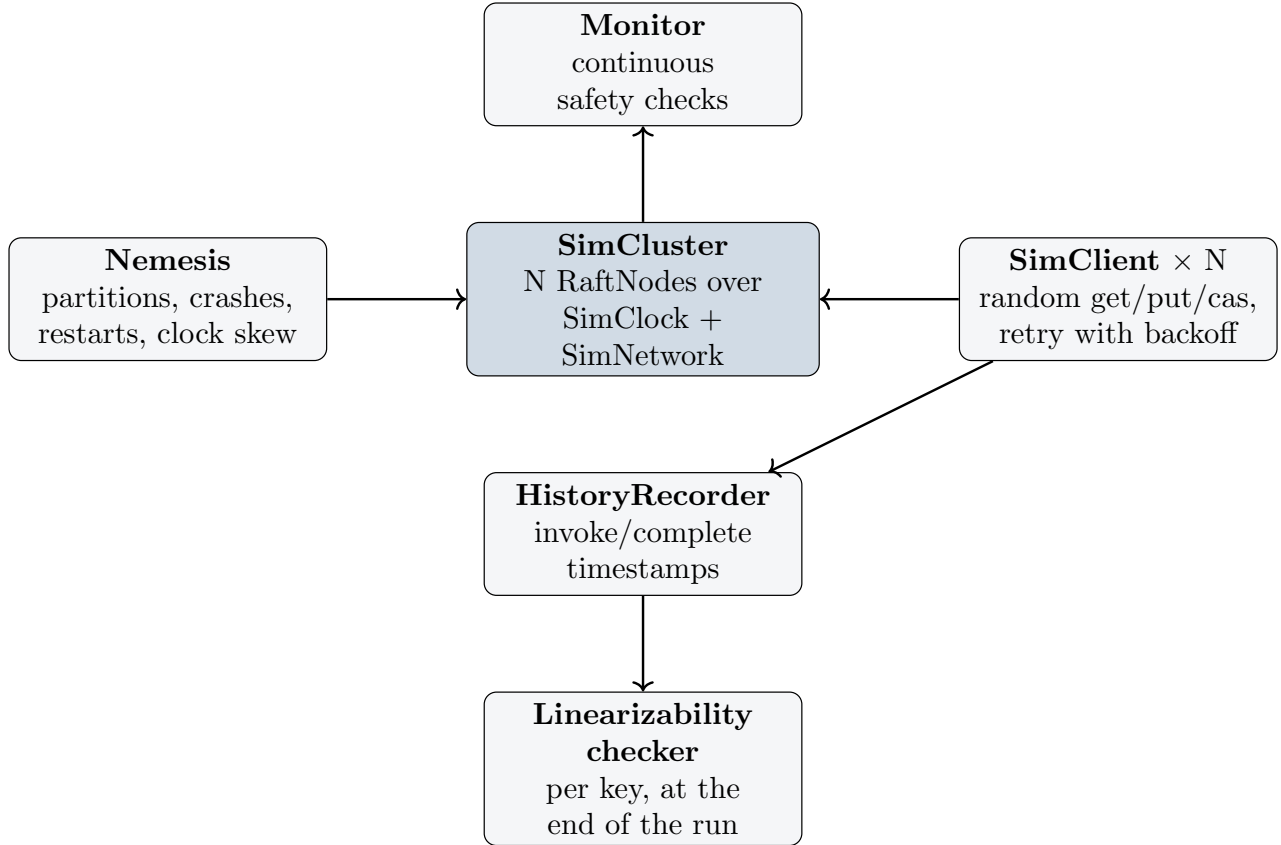


Figure 7: One seeded simulation run wires together a cluster, an adversarial nemesis, concurrent clients, a continuous safety monitor, and a post-hoc linearizability check on the recorded history.

- **Partition:** shuffle the node ids, cut the list at a random point, and cut all connectivity between the two resulting groups (healing any prior partition first, so partitions don’t silently stack).
- **Crash** a random currently-alive node (never the last one standing – `if len(alive) > 1`), scheduling its restart after a random delay.
- **Clock skew:** scale a random node’s timer speed by a factor between 0.5x and 1.8x.
- Otherwise: heal all partitions.

After the chaos window ends, `_stabilize` heals everything and restarts every crashed node, giving the last 30% of the run (called the “drain” window) a chance for in-flight client operations to actually complete before the run is scored.

7.2 Continuous safety invariants

Monitor checks four invariants throughout the run, each of which raises `InvariantViolation` (carrying the seed) the instant it is violated, rather than only at the end:

1. **Election safety** – at most one leader per term, ever. Tracked via `leaders_by_term`; a second distinct leader claiming the same term is an immediate failure.
2. **Leader completeness** – the instant a node wins an election, its log must already contain every entry that has ever been applied anywhere (i.e. a new leader can never be missing something the

Parameter	Default (from <code>SimConfig</code>)
Nodes	3
Duration	12.0 virtual seconds
Concurrent clients	3, over 3 keys
Message loss / duplication	5% / 5%
Message delay	1–30ms (uniform)
Nemesis action mix	35% partition, 25% crash, 15% clock skew, rest heal
Snapshot threshold	40 applied entries

Table 1: Default `SimConfig` values used by every seed in the sweep.

cluster already agreed on). Checked in `_check_leader_completeness` against a running record of every `(index, term, payload)` ever applied by any node.

3. **State machine safety** – if two different nodes ever apply different content at the same log index, that is an immediate failure (`self.applied` records the first thing applied at each index; a mismatch on a later apply fails immediately).
4. **Log matching** – periodically (every 200 simulated steps, `on_step`), every live pair of nodes is checked pairwise: if two logs have entries with equal index and equal term, those entries must be byte-identical (this is exactly Raft’s “Log Matching Property” from the paper, and it is what the whole `AppendEntries` consistency-check mechanism exists to guarantee).

The safety net is itself tested. `test_sim_invariants.py` includes `test_broken_raft_is_caught`, which deliberately weakens the commit rule (dropping the majority requirement) and asserts that the sweep now reliably detects the resulting split brain within a few dozen seeds. The point, stated directly in the README: “a checker or invariant monitor that never fires is indistinguishable from one that cannot fire,” so the monitor’s ability to actually catch a real bug is verified, not merely assumed.

7.3 Simulated clients

Each `SimClient` repeatedly picks a random key and, with roughly 40% probability, issues a `get`; 35%, a `put`; and the rest, a `cas` against a plausible prior value (or explicit absence). Each attempt targets a random node; if that node can’t currently accept the operation (not the leader, or the read/write is rejected), the client retries against a different node with jittered backoff, up to 8 attempts, before giving up and recording the operation as **incomplete**. Every invocation and (if one arrives) completion is timestamped into a shared `HistoryRecorder`, which is what the linearizability checker consumes after the run.

8 The Linearizability Checker (checker/)

8.1 The problem

A recorded **history** is a set of operations, each with an **invocation** time and, if a response arrived, a **completion** time and result. An operation whose response never arrived (`op.complete == math.inf`, i.e. `is_complete` is false) is **incomplete**: the client genuinely cannot tell whether it took effect on the server before the connection died, so a correct checker must consider both possibilities.

Checking a history for linearizability means asking: does there exist some way to order all the

operations into a single sequential timeline, consistent with real-time (an operation that visibly completed before another one was invoked must come first), such that applying them one at a time to a simple, single-threaded model of the object reproduces every result the clients actually observed? If yes, the concurrent, distributed system behaved exactly as if it were one un-contended in-memory object – the strongest, most intuitive correctness guarantee a KV store can offer.

8.2 Wing & Gong search with Lowe’s memoization

`checker/linearizability.py` implements this from scratch as a depth-first search over (**linearized-set, register value**) states, per the classical Wing & Gong algorithm with the memoization trick attributed to Lowe:

- Because every key in this KV store is an independent register (an operation on key "a" can never affect key "b"), `check_history` checks each key’s sub-history completely independently – by far the largest practical reduction in search space, since a chaos run’s operations are spread over several keys.
- Operations are represented as bits in an integer bitmask (one bit per operation, so “operation 3 has been linearized” is bit 3 being set), capping this implementation at 63 operations per key per history (a Python integer bit position limit chosen here, not a fundamental algorithmic one).
- At each search state, an operation is **minimal** (eligible to be linearized next) if no still-unlinearized operation’s *completion* time is before this operation’s *invocation* time – i.e. nothing that (real-time-provably) had to happen first is being skipped.
- Applying a candidate minimal operation to the current model register value (`_step`) must reproduce the client’s observed result (for a complete operation) or is simply accepted unconditionally (for an incomplete one, which is free to “not have happened” or “have happened with any outcome,” since the client never learned which).
- Each distinct (`mask, value`) pair reached is memoized (**seen**); revisiting an already-explored state is skipped. Lowe’s insight is that this memoization collapses what would otherwise be an exponential blow-up in practice, because many interleavings converge on the same (`mask, value`) state.
- If every *completed* operation gets linearized (leftover incomplete ones are free to be declared as never having happened), the history is linearizable: **Outcome.OK**. If the search exhausts all states without finding such a path, **Outcome.VIOLATION**. If the search exceeds `max_states` (2,000,000 by default) before resolving either way, the honest answer **Outcome.UNKNOWN** is returned – never a false OK and never a false VIOLATION.

Why this is fundamentally hard. Checking linearizability of an arbitrary history is NP-complete in the number of concurrent operations – the search space of possible interleavings grows combinatorially. This project doesn’t evade that; it manages it: per-key decomposition, memoization, and an explicit budget with an honest UNKNOWN outcome rather than either hanging forever or silently returning a wrong answer. The README notes chaos-run histories here (roughly 30–60 operations per key, with bounded concurrency because only a few clients run at once) verify in milliseconds; a deliberately adversarial, very wide history would not.

8.3 Guarding against a checker that always says yes

`test_linearizability.py` includes hand-built, known-bad synthetic histories – a **stale read** (a `get` returning an old value after a later `put` definitely completed), a **lost update** (two writes where a later one silently vanishes), and a **phantom value** (a `get` returning something no operation ever wrote) – and asserts the checker rejects every one of them with `VIOLATION`. This exists because a checker with a subtle bug that lets it silently accept everything, always returning `OK`, would be worse than no checker at all (it would look like a safety net while providing none). The same defensive-testing instinct as the Monitor’s weakened-quorum canary test in Section 6 appears here again.

The README also notes a specific bug caught in the checker’s own test suite: an early “checker budget” test used 40 incomplete concurrent puts, which the search resolves *instantly* by declaring none of them ever happened (a trivially linearizable, cheap answer) – so the test never actually exercised the exhaustive-search budget it was meant to test. The fix buries a real violation under 20 *completed* concurrent puts, forcing the full exhaustive search.

9 Wiring It All Together: `server.py`, `client.py`, `cli/`

9.1 RaftServer: one real cluster node

`RaftServer` (`server.py`) is what `raftkv server` actually runs: it constructs a `FileStorage`, a `KVMachine`, a `TcpTransport/AsyncioEnv` pair, and finally a `RaftNode` wired to all of them, then serves both Raft peer traffic and client requests over the one TCP listener.

Client requests map onto Raft as follows:

- `put/delete/cas` call `node.propose(cmd)`, which returns immediately with `(index, term)`; the server then awaits an `asyncio.Future` registered under that `(index, term)` key, resolved by `_on_apply` once the entry actually commits and applies (or resolved to `None` – reported to the client as `leadership_lost` – if the node steps down before that happens, via `_on_step_down`, or if a different entry ends up at that index because of a term change, handled by the “stale key” cleanup loop in `_on_apply`).
- `get` calls `node.read_index`, awaiting the resulting read index the same way, then serves straight from the local `KVMachine`.
- `add_server/remove_server` register the peer’s address with the transport (for adds) and call `node.add_server/remove_server`, then wait for that config entry to commit.
- `status` answers immediately from local node state and is never redirected to the leader – useful precisely because you want to be able to ask *any* node, including a partitioned or crashed one, what it currently believes.

Every write/read/config path also enforces a timeout (`asyncio.wait_for`, 5s for writes/config, 2s for reads) so a client is never left hanging indefinitely if the cluster becomes unavailable mid-request.

9.2 RaftClient: leader-hint following and retries

`client.py`’s `RaftClient` opens one fresh TCP connection per request (a known, stated inefficiency – see Section 11) and implements the retry loop every production Raft client needs:

- Every write allocates a strictly monotonically increasing `seq` from a per-client `itertools.count`, so retries are guaranteed deduplicable by the server’s session table (Section 4.6’s companion mech-

anism in `kv/machine.py`). The README recounts a real bug caught here during development: an early test client used *random* sequence numbers per retry, which the (correctly behaving) session table interpreted as “older, already-superseded write” and silently discarded – the fix, a monotonic counter, is now baked into the client library itself so this class of mistake can’t recur.

- If a node reports `not_leader`, the client follows its `leader` hint on the next attempt (falling back to round-robin across all known addresses if there is no hint, or the hint points back at the node that just rejected the request).
- `leadership_lost` and `read_timeout` responses clear the cached leader hint and retry, since the previous leader may no longer be one.
- The whole loop is bounded by `total_timeout` (15s default), after which `ClusterError` is raised.

9.3 The CLI

`cli/main.py` implements the `raftkv` command’s subcommands: `server` (run one node in the foreground, reading topology from a comma-separated `id=host:port` list, optionally starting with an empty config via `--join` to be added later); `cluster start/stop/status` (spawns/kills one real OS subprocess per node via `subprocess.Popen`, so `kill -9` on a node’s PID is a genuinely real crash, not a simulated one, and persists the topology to `$RAFTKV_HOME/cluster.json`); the KV verbs `put/get/del/cas`; and `sim`, which drives `sim/batch.py`’s multi-process seed sweep runner (`multiprocessing.Pool`, one OS process per worker, each running independent seeds via `run_simulation`) and prints the aggregate throughput and failure count shown in Section 10.

10 Testing and Verification

10.1 Test suites

Ten test modules under `tests/` exercise the system from multiple angles (all descriptions below reflect what `docs/TESTING.md` states these suites cover, cross-checked against the module names and docstrings present in the repository):

- `test_election.py`, `test_replication.py` – unit-level scenarios against the simulator with faults disabled or targeted, including the paper’s Figure 7 replication scenarios (the six canonical follower-log states a new leader might encounter) and the Section 5.4.2 commit restriction.
- `test_persistence.py` – `FileStorage` torn-write recovery, atomic hardstate writes, and the crash windows around snapshotting/log rewriting.
- `test_snapshot.py` – compaction thresholds, `InstallSnapshot` catch-up of a partitioned follower, restart from a snapshot.
- `test_membership.py` – add/remove server, leader self-removal, the one-change-at-a-time rule.
- `test_kv_sessions.py` – dedup semantics and exactly-once behavior across a failover.
- `test_readindex.py` – a stale leader stuck in a minority partition cannot serve reads.
- `test_sim_invariants.py` – determinism (identical seed reproduces an identical run), a quick sweep, the harness self-test (`test_broken_raft_is_caught`), and the 200-seed slow sweep.
- `test_linearizability.py` – the checker’s own known-bad synthetic histories (Section 8).

- `test_tcp_integration.py` – a real 3-node asyncio TCP cluster on localhost: put/get/cas, leader kill and failover, restart catch-up, full-cluster restart from disk.

10.2 A noteworthy testing challenge: Figure 7 without real elections

The paper’s Figure 7 scenarios describe specific follower log states relative to a leader that has already won an earlier term through a prior election. Two of the six cases (labeled (c) and (d) in the paper) describe followers whose logs are *more* up to date than the intended leader’s – meaning that in a small real cluster, those followers would simply deny the intended leader their votes, making it impossible to set up the scenario through an actual election. The project’s solution, stated in the README: install leadership directly (bypassing the election) and freeze the follower’s election timer, which reproduces the exact log-state scenario the figure describes without weakening or bypassing the vote-granting rule itself anywhere in production code.

10.3 Verification results actually observed

These are the exact figures reported in the project’s README, run on the author’s own machine (Linux, Python 3.12.3) – reproduced here verbatim, not recomputed or estimated:

- **pytest: 57 tests, all passing**, including the Figure 7 replication scenarios, torn-write recovery, membership changes over both transports, exactly-once failover retries, ReadIndex staleness tests, checker known-bad histories, the 200-seed slow sweep, and the real 3-node TCP integration tests.
- The documented long randomized run:

```
$ raftkv sim --seeds 5000 --workers 10
5000 seeds in 84.7s (59.0 seeds/s), 564170 client ops total, 0 failure(s)
```

Each of those 5000 seeds independently simulates 12 virtual seconds of a 3-node cluster under partitions, crashes, restarts, 5% message loss, 5% duplication, and election-timer clock skew, with every completed client operation checked for linearizability.

- The harness’s own teeth are verified: weakening the quorum rule in a deliberate test makes the sweep detect the resulting split brain within a few dozen seeds.

11 Honest Limits, Challenges, and Lessons Learned

Everything in this section is taken directly from the README’s “Honest limits,” “Challenges,” “What I learned,” and “What I’d do differently” sections – these are the author’s own stated assessments, not inferred by this document.

11.1 Stated honest limits

- Built and tested for single-digit node counts on localhost; throughput is bounded by Python, JSON serialization, and one `fsync` per append batch – explicitly a correctness project, not a fast one.
- The linearizability checker’s window: at most 63 operations per key per history, with an honest UNKNOWN rather than a false answer past its state budget; adversarial, very wide histories are out of scope.
- Snapshots ship as one whole message, not chunked.

- No pre-vote: a node that rejoins after a network partition heals forces one avoidable re-election (via term inflation) before settling back in quietly.
- Client sessions never expire, so the dedup table grows with the number of distinct client ids ever seen.
- A newly added server only starts replicating after its membership-change entry is appended (no pre-join catch-up rounds), so adding a far-behind server can briefly raise the effective commit quorum required.

11.2 Real challenges hit during development

- Python 3.12 changed `asyncio.Server.wait_closed()` to also wait for in-flight connection handlers to finish, which deadlocked server shutdown (client connections were blocked forever inside `readexactly`), hanging the TCP integration test for minutes at near-zero CPU. Fixed by tracking every accepted `StreamWriter` and explicitly closing them before awaiting `wait_closed()` (visible in `TcpTransport.stop`).
- The exactly-once dedup mechanism initially broke its own test harness: a test helper sent writes under one client id with *random* sequence numbers, and the (correctly behaving) session table discarded any write whose seq looked older than one already seen, silently dropping legitimate later writes and failing two unrelated tests. Fixed with a monotonic per-client counter, now permanently baked into `RaftClient`.
- A fresh leader cannot answer `ReadIndex` reads before its own no-op entry commits, since its `commit_index` may still reflect a stale view of what is truly durable; pending reads register with no read index and get one assigned automatically the moment the leader's own-term commit lands. This exact design was driven directly by a failover read test.
- Getting the simulator bit-for-bit deterministic required treating every source of randomness as explicit, seeded, threaded state (Section 6); the same-seed-twice fingerprint test caught two real instances of accidental reliance on Python's ambient iteration order.

11.3 What the author says they learned

- Sans-IO is described as “the whole ballgame” for testable distributed systems: once the consensus core stopped owning sockets and clocks, running 5000 chaos schedules in under 90 seconds became an ordinary function call, and every bug the simulator found was, by construction, a production bug.
- Exactly-once is framed as a state-machine feature, not a networking feature – the dedup table has to live *inside* the replicated state and its snapshots, or a leader failover silently downgrades the system back to at-least-once.
- Crash safety, in the author's words, “falls out of one discipline”: never acknowledge before `fsync`, and make every on-disk structure recoverable either by atomic rename or by scan-and-truncate. The torn-tail CRC scan is noted as being under 20 lines of code.
- Safety nets need their own tests – both the weakened-quorum canary test on the Monitor and the known-bad histories on the linearizability checker exist specifically because an invariant monitor or checker that never fires looks identical to one that cannot fire.

- The Raft paper’s Section 5.4.2 (the own-term commit restriction) and the ReadIndex term-commit precondition are singled out as the two places where a plausible-looking shortcut is actually a real linearizability bug – and the chaos harness is what found the second one, in the author’s own account.

11.4 What the author says they would change

- Replication is described as “naive”: the leader resends the whole remaining log suffix to a follower on every heartbeat instead of tracking in-flight byte ranges. Correct (`AppendEntries` is idempotent – resending already-applied entries does no harm) but wasteful; pipelining with a per-follower window would cut chattiness and also bound message sizes.
- `InstallSnapshot` re-reads the whole snapshot from disk on every send; a cached handle plus chunked transfer with an offset (as the dissertation describes) is called out as the obvious upgrade.
- Add pre-vote, to eliminate the avoidable re-election after a partition heals; the simulator is noted as able to verify such a fix within minutes once written.
- The simulator’s `MemoryStorage` models a perfect disk; injecting torn writes and `fsync` reordering into simulated storage would let chaos runs exercise recovery paths that today are only unit-tested.
- The CLI client opens a new TCP connection per request; a persistent, multiplexed connection is what the frame protocol was designed to support, and the author states this was not finished.
- Session expiry (a TTL agreed through the log, as in the dissertation) is stated as a prerequisite before the dedup table could be called production-ready.

12 Glossary

asyncio

Python’s built-in library for writing concurrent code using a single-threaded **event loop** that switches between tasks whenever one is waiting on I/O (like a network read), instead of using OS threads.

CRC32

A 32-bit checksum: a small number computed from a block of data such that almost any accidental change to the data changes the checksum too, letting corruption (like a torn write) be detected cheaply.

fsync

A system call that blocks until the operating system has actually pushed previously written data from memory buffers down to the physical storage device, so it will survive a power loss or crash.

Idempotent

An operation that has the same effect no matter how many times it is applied (applying it twice is the same as applying it once). `AppendEntries` is idempotent: resending already-known entries changes nothing.

JSON

JavaScript Object Notation: a simple, human-readable text format for structured data, used here for both wire messages and on-disk persistence.

Log entry

One unit in the replicated log: an index, a term, a kind (`noop/command/config`), and a payload.

Median of match_index

With N nodes' `match_index` values sorted, the value at position $N - \text{majority}$ from the start is exactly the highest index that a majority of nodes has all reached – this is how a leader computes the new commit index in one step without checking every possible majority combination.

Term

A monotonically increasing integer that identifies one “epoch” of the cluster, incremented every time a new election starts. Higher term always wins in any comparison; it is Raft’s way of totally ordering “who might be leader when.”

TTL

Time-to-live: an expiration deadline after which some piece of state (here, a hypothetical client session) is considered stale and can be discarded.